ELSEWEDY UNIVERSITY OF TECHNOLOGY
- POLYTECHNIC OF EGYPT -

Design and Analysis of Algorithms

Lecture 8: Dynamic Programming

© Fall 2025 – **Dr. Dalia Ezzat**
dalia.ezzat@sut.edu.eg

Table of Contents

Dynammic Programming

- Dynamic programming (DP) is a powerful technique used to solve complex problems by breaking them down into simpler subproblems.

- Invented by American mathematician Richard Bellman in the 1950s to solve optimization problems and later assimilated by CS.

- “Programming” here means “planning”.

- Two of the most common approaches in dynamic programming are **memoization** and **tabulation**. Both methods aim to optimize the efficiency of algorithms by storing the results of subproblems to avoid redundant computations. However, they differ in their implementation and use cases.

Dynammic Programming- Memoization

- Memoization is a top-down approach to dynamic programming. It involves solving a problem by recursively breaking it down into smaller subproblems and storing the results of these subproblems to avoid recomputing them.

- When a subproblem is solved, its result is stored in a data structure, typically a hash table or an array, so that it can be reused if the same subproblem is encountered again.

Dynammic Programming- Memoization

- **How Memoization Works**

  - ✓ **Recursive Function:** The problem is solved using a recursive function that calls itself for smaller subproblems.

  - ✓ **Storage of Results:** Before making a recursive call, the algorithm checks if the result of the subproblem is already stored in the memoization table. If it is, the stored result is returned instead of recalculating it.

  - ✓ **Base Case:** The recursive function includes a base case to terminate the recursion.

  - ✓ **Store and Return:** If the subproblem result is not stored, the recursive function computes it, stores it in the memoization table, and then returns the result.

Dynammic Programming- Tabulation

- Tabulation is a bottom-up approach to dynamic programming. Instead of solving the problem recursively, it builds a table iteratively from the base cases up to the desired solution.

- The idea is to solve all subproblems in a systematic manner and store their results in a table (usually an array) to use them in solving larger subproblems.

Dynammic Programming- Tabulation

- **How Tabulation Works**

✓ **Initialize the Table:** A table (usually an array) is created to store the results of subproblems.

✓ **Base Case:** The base cases are initialized directly in the table.

✓ **Iterative Computation:** The table is filled iteratively, starting from the smallest subproblems and building up to the desired problem.

✓ **Final Solution:** The solution to the original problem is found in the last cell of the table.

Dynamic Programming Vs. Divide and Conquer

- **Similarity:** Both techniques break a problem into smaller subproblems.
- **Crucial Difference:**
  - **Divide and Conquer:** Subproblems are **independent**. Solving one doesn't require the solution of another.
  - **Dynamic Programming (DP):** Subproblems are **overlapping**. They are interdependent, and the same subproblem is solved multiple times in a naive recursive approach. DP eliminates this redundancy.
  - **Divide and conquer:** top-down
  - **Dynamic programming:** bottom-up

Dynamic Programming Vs. Greedy Algorithms

- **Similarity:** Both are used for **optimization problems** (finding a maximum or minimum).

- **Crucial Difference:**

    - **Greedy:** Makes the locally optimal choice at each step, hoping it leads to a global optimum. It's fast but **does not guarantee** an optimal solution for all problems.

    - **Dynamic Programming:** Considers all possibilities at each step. It makes decisions based on solutions to smaller subproblems, which **guarantees** finding the optimal solution.

Some Problems Solved with Dynamic Programming

- Knapsack problems (we'll do 0/1)
- Longest common subsequence

ELSEWEDY UNIVERSITY OF TECHNOLOGY
- POLYTECHNIC OF EGYPT -

Let's revisit the 0/1 knapsack problem from the previous lecture which the greedy algorithm fails to find the optimal solution. You're a thief with a knapsack that can carry specific amount of goods

The Naïve Algorithm: Brute Force

Dynamic Programming

0/1 Knapsack problem using brute force

- Consider all possible subsequences, the calculates the value for each subsequence, finally take the maximum value:

(10), (20), (30), (10+20), (10+30), (20+30), (10+20+30)

- So, the solution is such as 1 0 0, 0 1 0, 0 0 1, 1 1 0, 1 0 1 ...

- Number of possible solution is dramatically increasing with increasing the input size then we have O(2^n).

0/1 Knapsack problem using dynamic programming

- You have A **knapsack** (backpack) that can hold up to **W = 5 kg.** Several **items** you can steal, each with: a **weight** (w_i), and a **value/benefit** (b_i). Goal is to choose items to put in your knapsack to **maximize total value** without exceeding the weight limit. **Constraint:** For each item, you either **take it (1)** or **leave it (0)** - no fractions. Available values are as follows:

Item	Weight	Value
1	2	3
2	3	4
3	4	5
4	5	6

0/1 Knapsack problem using dynamic programming

- Every dynamic-programming algorithm (tabulation) starts with a table of size (# of items+1, Capacity +1). Here's a table for the knapsack problem.

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1						
2						
3						
4						

```
for w = 0 to W
        B[0,w] = 0
```

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0					
2	0					
3	0					
4	0					

```
for i = 1 to n
        B[i,0] = 0
```

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0				
2	0					
3	0					
4	0					

$i=1$

$b_i=3$

$w_i=2$

$w=1$

$w-w_i=-1$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i <= w // item i can be part of the solution
    if $b_i + B[i-1,w-w_i] > B[i-1,w]$
        $B[i,w] = b_i + B[i-1,w- w_i]$
    else
        $B[i,w] = B[i-1,w]$
else **B[i,w] = B[i-1,w]** // $w_i > w$

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	**3**			
2	0					
3	0					
4	0					

i=1

b_i=3

w_i=2

w=2

w-w_i =0

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i <= w // item i can be part of the solution

if b_i + B[i-1,w-w_i] > B[i-1,w]

B[i,w] = b_i + B[i-1,w- w_i]

else

B[i,w] = B[i-1,w]

else B[i,w] = B[i-1,w] // w_i > w

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	**3**		
2	0					
3	0					
4	0					

$i=1$

$b_i=3$

$w_i=2$

$w=3$

$w-w_i=1$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i <= w // item i can be part of the solution

  if $b_i + B[i-1,w-w_i] > B[i-1,w]$

    $B[i,w] = b_i + B[i-1,w- w_i]$

  else

    $B[i,w] = B[i-1,w]$

else $B[i,w] = B[i-1,w]$ // $w_i > w$

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	**3**	
2	0					
3	0					
4	0					

i=1

b_i=3

w_i=2

w=4

w-w_i =2

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i <= w // item i can be part of the solution

if b_i + B[i-1,w-w_i] > B[i-1,w]

B[i,w] = b_i + B[i-1,w- w_i]

else

B[i,w] = B[i-1,w]

else B[i,w] = B[i-1,w] // w_i > w

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	**3**
2	0					
3	0					
4	0					

$i=1$

$b_i=3$

$w_i=2$

$w=5$

$w-w_i=3$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i <= w$ // item i can be part of the solution

  if $b_i + B[i-1,w-w_i] > B[i-1,w]$

    $B[i,w] = b_i + B[i-1,w- w_i]$

  else

    $B[i,w] = B[i-1,w]$

else $B[i,w] = B[i-1,w]$ // $w_i > w$

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	**0**				
3	0					
4	0					

i=2

b_i=4

w_i=3

w=1

w-w_i =-2

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i <= w // item i can be part of the solution

if b_i + B[i-1,w-w_i] > B[i-1,w]

B[i,w] = b_i + B[i-1,w- w_i]

else

B[i,w] = B[i-1,w]

else **B[i,w] = B[i-1,w]** // w_i > w

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	**0**				
3	0					
4	0					

i=2

b_i=4

w_i=3

w=1

w-w_i =-2

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i <= w // item i can be part of the solution

    if b_i + B[i-1,w-w_i] > B[i-1,w]

        B[i,w] = b_i + B[i-1,w- w_i]

    else

        B[i,w] = B[i-1,w]

else **B[i,w] = B[i-1,w]** // w_i > w

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	**3**			
3	0					
4	0					

$i=2$

$b_i=4$

$w_i=3$

$w=2$

$w-w_i=-1$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i <= w$ // item i can be part of the solution

    if $b_i + B[i-1,w-w_i] > B[i-1,w]$

        $B[i,w] = b_i + B[i-1,w- w_i]$

    else

        $B[i,w] = B[i-1,w]$

else **B[i,w] = B[i-1,w]** // $w_i > w$

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	**4**		
3	0					
4	0					

$i=2$

$b_i=4$

$w_i=3$

$w=3$

$w-w_i=0$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i <= w$ // item i can be part of the solution

  if $b_i + B[i-1,w-w_i] > B[i-1,w]$

    $B[i,w] = b_i + B[i-1,w- w_i]$

  else

    $B[i,w] = B[i-1,w]$

else $B[i,w] = B[i-1,w]$ // $w_i > w$

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	**4**	
3	0					
4	0					

$i=2$

$b_i=4$

$w_i=3$

$w=4$

$w-w_i=1$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i <= w$ // item i can be part of the solution

  if $b_i + B[i-1,w-w_i] > B[i-1,w]$

    $B[i,w] = b_i + B[i-1,w- w_i]$

  else

    $B[i,w] = B[i-1,w]$

else $B[i,w] = B[i-1,w]$ // $w_i > w$

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0					
4	0					

$i=2$

$b_i=4$

$w_i=3$

$w=5$

$w-w_i=2$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i <= w$ // item i can be part of the solution

  if $b_i + B[i-1,w-w_i] > B[i-1,w]$

    $B[i,w] = b_i + B[i-1,w- w_i]$

  else

    $B[i,w] = B[i-1,w]$

else $B[i,w] = B[i-1,w]$ // $w_i > w$

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	**0**	**3**	**4**		
4	0					

$i=3$

$b_i=5$

$w_i=4$

$w= 1..3$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i <= w // item i can be part of the solution
    if $b_i + B[i-1,w-w_i] > B[i-1,w]$
        $B[i,w] = b_i + B[i-1,w- w_i]$
    else
        $B[i,w] = B[i-1,w]$
else **B[i,w] = B[i-1,w]** // $w_i > w$

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	**0**	**3**	**4**		
4	0					

$i=3$

$b_i=5$

$w_i=4$

$w= 1..3$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i <= w // item i can be part of the solution
    if $b_i + B[i-1,w-w_i] > B[i-1,w]$
        $B[i,w] = b_i + B[i-1,w- w_i]$
    else
        $B[i,w] = B[i-1,w]$
else **B[i,w] = B[i-1,w]** // $w_i > w$

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	**5**	
4	0					

i=3

b_i=5

w_i=4

w= 4

w- w_i=0

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i <= w // item i can be part of the solution

if b_i + B[i-1,w-w_i] > B[i-1,w]

B[i,w] = b_i + B[i-1,w- w_i]

else

B[i,w] = B[i-1,w]

else B[i,w] = B[i-1,w] // w_i > w

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0					

$i=3$

$b_i=5$

$w_i=4$

$w= 5$

$w- w_i=1$

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

if w_i <= w // item i can be part of the solution
    if $b_i + B[i-1,w-w_i] > B[i-1,w]$
        $B[i,w] = b_i + B[i-1,w- w_i]$
    else
        B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w] // $w_i > w$

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	**0**	**3**	**4**	**5**	

$i=4$

$b_i=6$

$w_i=5$

$w= 1..4$

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

if w_i <= w // item i can be part of the solution
    if $b_i + B[i-1,w-w_i] > B[i-1,w]$
        $B[i,w] = b_i + B[i-1,w- w_i]$
    else
        $B[i,w] = B[i-1,w]$
else **B[i,w] = B[i-1,w]** // $w_i > w$

0/1 Knapsack problem using dynamic programming

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

$i=4$

$b_i=6$

$w_i=5$

$w= 5$

$w- w_i=0$

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

if w_i <= w // item i can be part of the solution
    if $b_i + B[i-1,w-w_i] > B[i-1,w]$
        $B[i,w] = b_i + B[i-1,w- w_i]$
    else
        B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w] // $w_i > w$

0/1 Knapsack problem using dynamic programming

- This algorithm only finds the max possible value that can be carried in the knapsack.

  ➢ The value in $B[n, W]$

- To know the items that make this maximum value, an addition to this algorithm is necessary.

0/1 Knapsack problem using dynamic programming

- **How to find actual Knapsack Items:**

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

i=4
k= 5
b_i=6
w_i=5
$B[i,k] = 7$
$B[i-1,k] = 7$

Items:
1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

i=n, k=W
while i,k > 0
    if $B[i,k] \neq B[i-1,k]$ then
        mark the i^{th} item as in the knapsack
        $i = i-1$, $k = k-w_i$
    else
        $i = i-1$

0/1 Knapsack problem using dynamic programming

- **How to find actual Knapsack Items:**

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

i=4
k= 5
b_i=6
w_i=5
$B[i,k] = 7$
$B[i-1,k] = 7$

Items:
1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

i=n, k=W
while i,k > 0
    if $B[i,k] \neq B[i-1,k]$ then
        mark the i^{th} item as in the knapsack
        $i = i-1, k = k-w_i$
    else
        $i = i-1$

0/1 Knapsack problem using dynamic programming

- **How to find actual Knapsack Items:**

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

i=3
k= 5
b_i=6
w_i=4
$B[i,k] = 7$
$B[i-1,k] = 7$

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

i=n, k=W
while i,k > 0
    if $B[i,k] \neq B[i-1,k]$ then
        mark the i^{th} item as in the knapsack
        $i = i-1, k = k-w_i$
    else
        $i = i-1$

0/1 Knapsack problem using dynamic programming

- **How to find actual Knapsack Items:**

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

i=2
k= 5
b_i=4
w_i=3
$B[i,k] = 7$
$B[i-1,k] = 3$
$k - w_i = 2$

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

i=n, k=W
while i,k > 0
    if $B[i,k] \neq B[i-1,k]$ then
        mark the i^{th} item as in the knapsack
        $i = i-1, k = k-w_i$
    else
        $i = i-1$

0/1 Knapsack problem using dynamic programming

- **How to find actual Knapsack Items:**

Items:
1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

i=1
k= 2
b_i=3
w_i=2
$B[i,k] = 3$
$B[i-1,k] = 0$
$k - w_i = 0$

i=n, k=W
while i,k > 0
    if $B[i,k] \neq B[i-1,k]$ then
        mark the i^{th} item as in the knapsack
        $i = i-1, k = k-w_i$
    else
        $i = i-1$

0/1 Knapsack problem using dynamic programming

- **How to find actual Knapsack Items:**

i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

i=0
k= 0

Items:

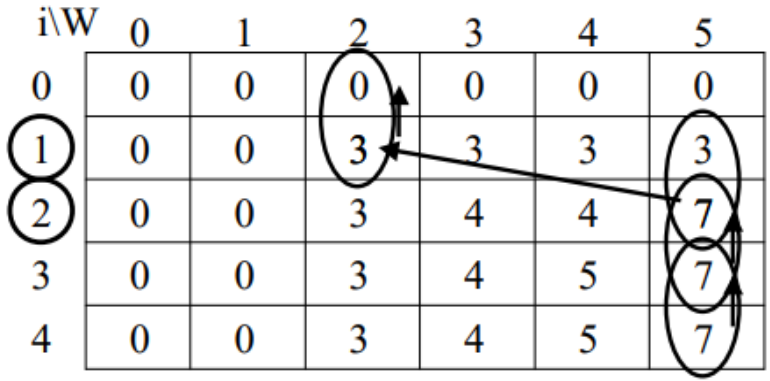
1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

The optimal knapsack should contain {1, 2}

i=n, k=W
while i,k > 0
    if $B[i,k] \neq B[i-1,k]$ then
        mark the n^{th} item as in the knapsack
        $i = i-1, k = k-w_i$
    else
        $i = i-1$

0/1 Knapsack problem using dynamic programming

- **How to find actual Knapsack Items:**



i\W	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

The optimal knapsack should contain {1, 2}

```
i=n, k=W
while i,k > 0
    if B[i,k] ≠ B[i−1,k] then
        mark the n^th item as in the knapsack
        i = i−1, k = k-w_i
    else
        i = i−1
```

0/1 Knapsack problem using dynamic programming

- **How to find actual Knapsack Items:**

All of the information we need is in the table.

$B[n,W]$ is the maximal value of items that can be placed in the Knapsack.

Let i=n and k=W

if $B[i,k] \neq B[i-1,k]$ then

  mark the i^{th} item as in the knapsack

  $i = i-1$, $k = k\text{-}w_i$

else

  $i = i-1$ // Assume the i^{th} item is <u>not</u> in the knapsack

  // Could it be in the optimally packed knapsack?

0/1 Knapsack problem using dynamic programming

- **The time complexity of the 0/1 Knapsack problem using dynamic programming is O(NW)*, where:**

  N: represents the number of items available.

  W: represents the maximum capacity of the knapsack.

ELSEWEDY UNIVERSITY OF TECHNOLOGY
- POLYTECHNIC OF EGYPT -

Given two strings, find the length of their longest common subsequence (a sequence that appears in both strings in the same relative order, but not necessarily adjacent).

The Naïve Algorithm: Brute Force

Dynamic Programming

LCS using Brute Force

- What is the Longest Common Subsequence of X and Y?

X = A **B** **C** **B**

Y = **B** D **C** A **B**

- **The brute-force method for LCS works as follows:**
  - Generate all possible subsequences of sequence X.
  - Generate all possible subsequences of sequence Y.
  - Compare every subsequence of X with every subsequence of Y.
  - Find the longest subsequence that appears in both lists.
  - **Time Complexity:** $O(2^M \times 2^N) = O(2^{M+N})$, where M and N are the lengths of the strings. For M=7, N=5, this is $2^{12} = 4096$ comparisons in the worst case.

LCS using DP

- What is the Longest Common Subsequence of X and Y?
- X = A **B** **C** **B**
- Y = **B** D **C** A **B**
- LCS(X, Y) = BCB

LCS using DP

i		j	0	1	2	3	4	5
			Yj	**B**	**D**	**C**	**A**	**B**
0	Xi							
1	**A**							
2	**B**							
3	**C**							
4	**B**							

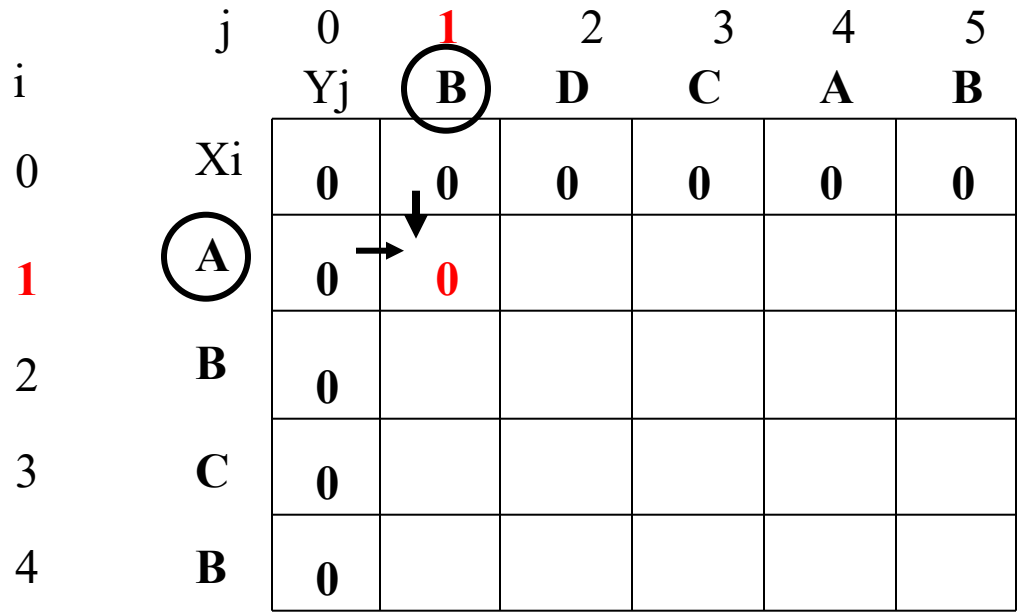
X = ABCB; m = |X| = 4
Y = BDCAB; n = |Y| = 5
Allocate array c[5,4]

LCS using DP

i	j	0	1	2	3	4	5
		Yj	**B**	**D**	**C**	**A**	**B**
0	Xi	**0**	**0**	**0**	**0**	**0**	**0**
1	**A**	**0**					
2	**B**	**0**					
3	**C**	**0**					
4	**B**	**0**					

for i = 1 to m c[i,0] = 0
for j = 1 to n c[0,j] = 0

LCS using DP



if ($X_i == Y_j$)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS using DP

i	j	0	1	2	3	4	5
		Yj	B	D	C	A	B
0	Xi	0	0	0	0	0	0
1	A	0	0	0	0		
2	B	0					
3	C	0					
4	B	0					

if ($X_i == Y_j$)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS using DP

i	j	0	1	2	3	4	5
		Yj	B	D	C	A	B
0	Xi	0	0	0	0	0	0
1	A	0	0	0	0	1	
2	B	0					
3	C	0					
4	B	0					

if ($X_i == Y_j$)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS using DP

i		j	0	1	2	3	4	5
			Yj	B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0	0	0	1 →	1
2	B		0					
3	C		0					
4	B		0					

if ($X_i == Y_j$)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS using DP

i		j	0	1	2	3	4	5
			Yj	B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1				
3	C		0					
4	B		0					

if ($X_i == Y_j$)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS using DP

i		j	0	1	2	3	4	5
			Yj	B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	
3	C		0					
4	B		0					

if ($X_i == Y_j$)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS using DP

i		j	0	1	2	3	4	5
			Yj	B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	2
3	C		0					
4	B		0					

if ($X_i == Y_j$)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS using DP

i		j	0	1	2	3	4	5
			Yj	B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	2
3	C		0	1	1			
4	B		0					

if ($X_i == Y_j$)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS using DP

i	j	0	1	2	3	4	5
		Yj	B	D	C	A	B
0	Xi	0	0	0	0	0	0
1	A	0	0	0	0	1	1
2	B	0	1	1	1	1	2
3	C	0	1	1	2		
4	B	0					

if ($X_i == Y_j$)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS using DP

i		j	0	1	2	3	4	5
			Yj	B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	2
3	C		0	1	1	2	2	2
4	B		0					

if ($X_i == Y_j$)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS using DP

i		j	0	1	2	3	4	5
			Yj	B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	2
3	C		0	1	1	2	2	2
4	B		0	1				

if ($X_i == Y_j$)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS using DP

i		j	0	1	2	3	4	5
			Yj	B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	2
3	C		0	1	1	2	2	2
4	B		0	1	1	2	2	

if ($X_i == Y_j$)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS using DP

i		j	0	1	2	3	4	5
			Yj	B	D	C	A	B
0	Xi		0	0	0	0	0	0
1	A		0	0	0	0	1	1
2	B		0	1	1	1	1	2
3	C		0	1	1	2	2	2
4	B		0	1	1	2	2	3

if ($X_i == Y_j$)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

LCS using DP

- So far, we have just found the *length* of LCS, but not LCS itself.
- Remember that:

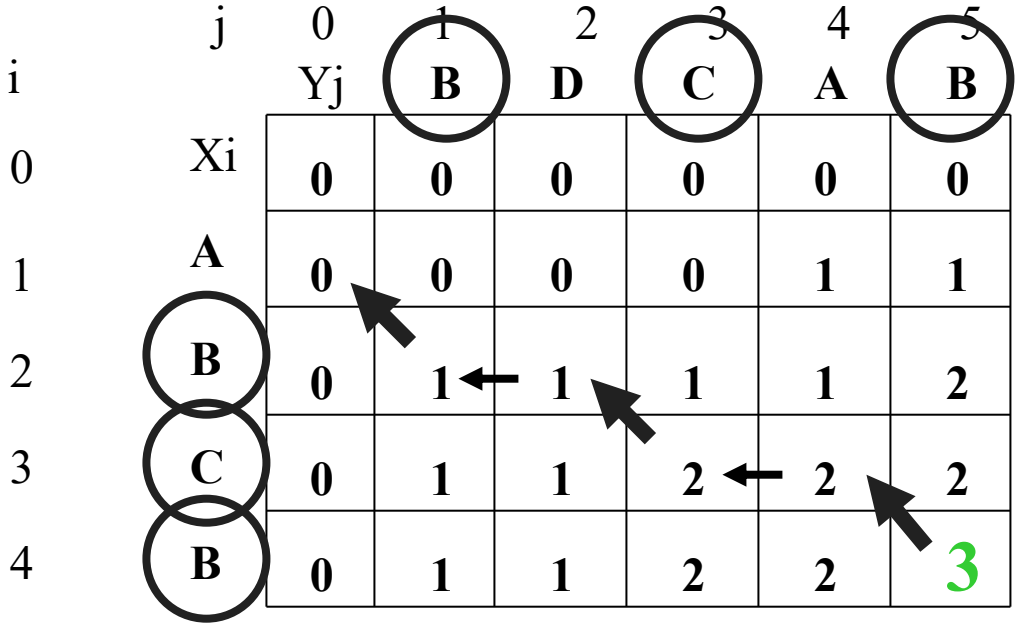
$$c[i,j] = \begin{cases} c[i-1,j-1]+1 & \text{if } x[i] = y[j], \\ \max(c[i,j-1], c[i-1,j]) & \text{otherwise} \end{cases}$$

- So we can start from *c[m,n]* and go backwards.
- When $i = 0$ or $j = 0$ (i.e. we reached the beginning), output remembered letters in reverse order.

LCS using DP

i	j	0	1	2	3	4	5
		Yj	**B**	**D**	**C**	**A**	**B**
0	Xi	**0**	**0**	**0**	**0**	**0**	**0**
1	**A**	**0**	**0**	**0**	**0**	**1**	**1**
2	**B**	**0**	**1**	**1**	**1**	**1**	**2**
3	**C**	**0**	**1**	**1**	**2**	**2**	**2**
4	**B**	**0**	**1**	**1**	**2**	**2**	**3**

LCS using DP



LCS (reversed order): **B C B**

LCS (straight order): **B C B**

LCS using DP

- LCS algorithm calculates the values of each entry of the array $c[m, n]$.
- So, what is the running time?
  - $O(m * n)$
  - Since each $c[i, j]$ is calculated in constant time, and there are m*n elements in the array.

```
1.  m ← length[X]
2.  n ← length[Y]
3.
4.  // Initialize tables
5.  for i ← 0 to m do
6.      c[i, 0] ← 0
7.  for j ← 0 to n do
8.      c[0, j] ← 0
9.
10. // Fill the DP tables
11. for i ← 1 to m do
12.     for j ← 1 to n do
13.         if x_i = y_j then
14.             c[i, j] ← c[i-1, j-1] + 1
15.             b[i, j] ← "↖"  // Characters match, move diagonally
16.         else if c[i-1, j] ≥ c[i, j-1] then
17.             c[i, j] ← c[i-1, j]
18.             b[i, j] ← "↑"  // Move upward (skip character from X)
19.         else
20.             c[i, j] ← c[i, j-1]
21.             b[i, j] ← "←"  // Move leftward (skip character from Y)
22.
23. return c and b
```

ELSEWEDY UNIVERSITY OF TECHNOLOGY
- POLYTECHNIC OF EGYPT -

Any Question?